

**DEPARTMENT OF TELECOMMUNICATION ENGINEERING BACHELOR OF
SCIENCE IN CYBER SECURITY
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO
Network Protocols & Architecture LAB
EXPERIMENT # 15**

Name: Muhammad Usman Roll No: 23BSCYS006

Score: _____ Signature of the Lab Tutor: _____ Date: _____

Getting Familiar with ACL

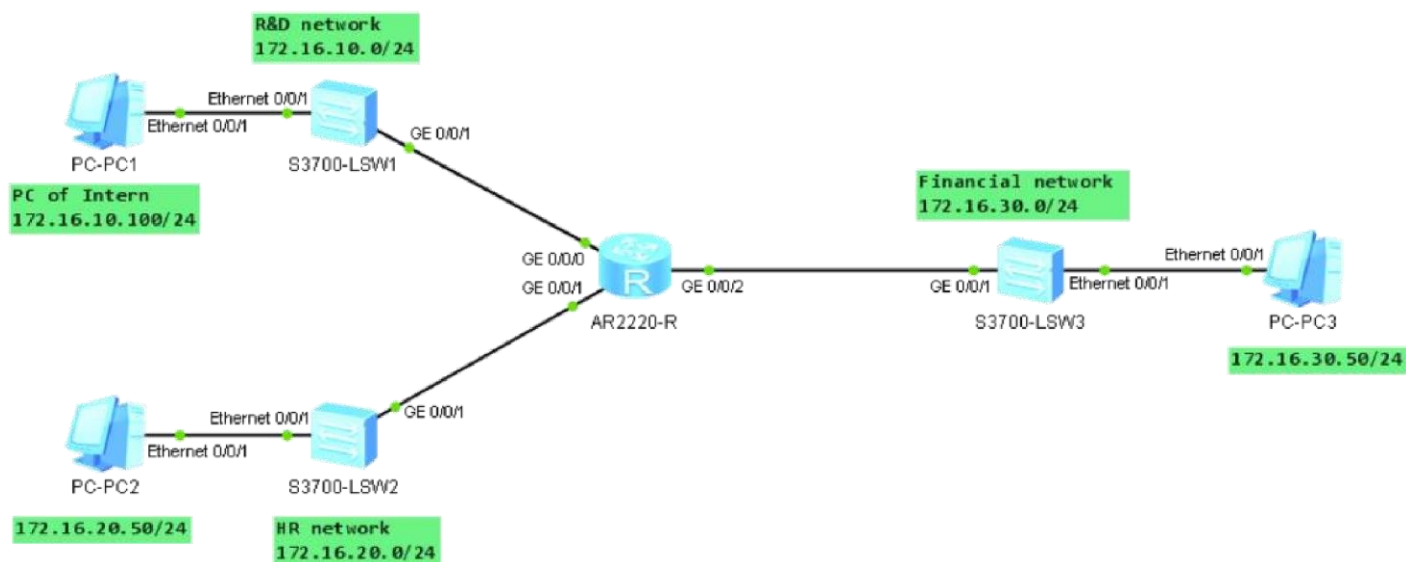
LAB PERFORMANCE INDICATOR	SUBJECT KNOWLEDGE	DATA ANALYSIS AND INTERPRETATION	ABILITY TO CONDUCT EXPERIMENT	SCORE
OBJECTIVE NO:				

Objective:

- Establishment of basic ACL to implement filtering [Learning Objectives:](#)

As a result of this lab section, you should achieve the following tasks:

- Establishment of a basic ACL to implement source-based filtering.



Topology

**DEPARTMENT OF TELECOMMUNICATION ENGINEERING BACHELOR OF
SCIENCE IN CYBER SECURITY
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO
Network Protocols & Architecture LAB
EXPERIMENT # 15**

Scenario

As shown in Figure, a company network includes the Research and Development (R&D), Human Resources, and Financial Department areas. In the R&D area, an intern is using a PC (IP address: **172.16.10.100/24**). For security reasons, we need to prevent the intern from sending packets to the Financial Department area. To do this, we can create a basic ACL on router R. The basic ACL will match IP packets sent from the intern's IP address. The ACL will be configured on **GE 0/0/2** and applied in the outbound direction.

Access Control List Fundamentals

An ACL consists of multiple deny or permit statements. When an ACL is configured on a device, the device compares received packets against the ACL rules one by one. If a packet matches an ACL rule, the device performs the action (permit or deny) as defined in the rule. When the packet matches a rule, the device stops the comparison process. If the packet does not match a rule, the packet is compared against the next rule. If the packet does not match any rule in the ACL, the device allows the packet, by default. This is known as an implicit permit. ACLs are divided into different types: basic, advanced, Layer-2, and user-defined. Each ACL configured on a network device has an ACL number. The range of possible ACL numbers is different depending on ACL type:

- **Basic:** 2000 to 2999
- **Advanced:** 3000 to 3999
- **Layer-2:** 4000 to 4999
- **User-defined:** 5000 to 5999

Each ACL rule has rule ID. By default, packets are matched against the rules in ascending order of rule IDs. By default, devices automatically assign an ID incrementally as they are created. For example, if the increment is 5 (Huawei device default), the automatically assigned rules will be 5, 10, 15, and so on. Likewise, if the rule ID increment is set to 2, the automatically assigned rule IDs will be 2, 4, 6, and so on. The larger the increment, the more rules that can be inserted between two adjacent rules.

The command syntax of a basic ACL is:

rule [**rule-id**] {**deny**|**permit**} [**source** {**source-address** **source-wildcard** | **any**}] [**fragment**] [**logging**] [**time-range** **time-name**]

- **rule:** Creates a rule
- **rule-id:** Rule ID number
- **deny|permit:** The rule either denies or permits as specified
- **source:** Specifies the source IP address
- **source-address:** Source IP address

**DEPARTMENT OF TELECOMMUNICATION ENGINEERING BACHELOR OF
SCIENCE IN CYBER SECURITY
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO
Network Protocols & Architecture LAB
EXPERIMENT # 15**

- **source-wildcard**: A wildcard in the source IP address. **source-wildcard** and **source-address** can be used together to determine a set of IP addresses or a single IP address. The usage of **source-wildcard** is the same as the usage of **wildcard-mask**
- **any**: Indicates that the source IP address can be any IP address
- **fragment**: Specifies that only non-first IP fragments can match this rule
- **logging**: Specifies that IP packets matching this rule will be logged
- **time-range** time-name: Specifies that the rule is effective only during the period of time-name.

Tasks

Step 1 Preparing the environment

```
<Huawei> system-view
Enter system view, return user view with Ctrl+Z.
[Huawei] sysname R
```

Step 2 Configure Router R

Configure the router interfaces to have IP addresses.

```
[R] interface GigabitEthernet 0/0/0
[R-GigabitEthernet0/0/0] ip address 172.16.10.1 24
[R-GigabitEthernet0/0/0] q
[R] interface GigabitEthernet 0/0/1
[R-GigabitEthernet0/0/1] ip address 172.16.20.1 24
[R-GigabitEthernet0/0/1] q
[R] interface GigabitEthernet 0/0/2
[R-GigabitEthernet0/0/2] ip address 172.16.30.1 24
[R-GigabitEthernet0/0/2] q
```

Now let's see how to configure router R. First, we need to create an ACL numbered 2000 in the system view of router R.

```
[R] acl 2000
[R-acl-basic-2000]
```

Then, in the ACL view, we create the following rule:

```
[R-acl-basic-2000] rule deny source 172.16.10.100 0.0.0.0
[R-acl-basic-2000] q
```

**DEPARTMENT OF TELECOMMUNICATION ENGINEERING BACHELOR OF
SCIENCE IN CYBER SECURITY
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO
Network Protocols & Architecture LAB
EXPERIMENT # 15**

This rule will deny the IP packets from 172.16.10.100
Run the display acl 2000 command to check the configuration of ACL 2000.

```
<R> display acl 2000
```

Finally, we use the **traffic-filter** command to apply ACL 2000 on the GE 0/0/2 interface of router R in the outbound direction.

```
[R] interface GigabitEthernet 0/0/2  
[R-GigabitEthernet0/0/2] traffic-filter outbound acl 2000  
[R-GigabitEthernet0/0/2] q  
[R]
```

Now, IP packets from 172.16.10.100 cannot pass through the GE 0/0/2 interface of router R.

Step 3 Configure PCs

Configure IP address on PCs.

**DEPARTMENT OF TELECOMMUNICATION ENGINEERING BACHELOR OF
SCIENCE IN CYBER SECURITY
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO
Network Protocols & Architecture LAB
EXPERIMENT # 15**

Host Name:

MAC Address:

IPv4 Configuration

☒ Static ☐ DHCP ☐ Obtain DNS server address automatically

IP Address: DNS1:

Subnet Mask: DNS2:

Gateway:

IPv6 Configuration

☒ Static ☐ DHCPv6

IPv6 Address:

Prefix Length:

IPv6 Gateway:

Host Name:

MAC Address:

IPv4 Configuration

☒ Static ☐ DHCP ☐ Obtain DNS server address automatically

IP Address: DNS1:

Subnet Mask: DNS2:

Gateway:

IPv6 Configuration

☒ Static ☐ DHCPv6

IPv6 Address:

Prefix Length:

IPv6 Gateway:

Host Name:

MAC Address:

IPv4 Configuration

☒ Static ☐ DHCP ☐ Obtain DNS server address automatically

IP Address: DNS1:

Subnet Mask: DNS2:

Gateway:

IPv6 Configuration

☒ Static ☐ DHCPv6

IPv6 Address:

Prefix Length:

IPv6 Gateway:

**DEPARTMENT OF TELECOMMUNICATION ENGINEERING BACHELOR OF
SCIENCE IN CYBER SECURITY
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO
Network Protocols & Architecture LAB
EXPERIMENT # 15**

Step 4 Verify the connectivity

```
PC> ipconfig

Link local IPv6 address.....: fe80::5689:98ff:fed6:6e57
IPv6 address.....: :: / 128  IPv6
gateway.....: ::
IPv4 address.....: 172.16.10.100
Subnet mask.....: 255.255.255.0
Gateway.....: 172.16.10.1
Physical address.....: 54-89-98-D6-6E-57
DNS server.....:
PC> ping 172.16.30.50
Ping 172.16.30.50: 32 data bytes, Press Ctrl_C to break  Request
timeout!
Request timeout!
Request timeout!
Request timeout!
Request timeout!

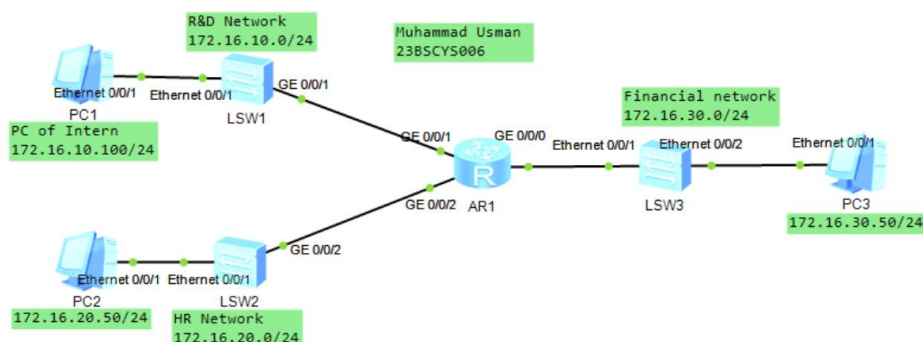
--- 172.16.30.50 ping statistics ---
    5 packet(s) transmitted
    0 packet(s) received
  100.00% packet loss
```

Tasks

Try a different topology and demonstrate your learning.

Upload the `.topo` `.efz` `.cfg` files

**DEPARTMENT OF TELECOMMUNICATION ENGINEERING BACHELOR OF
SCIENCE IN CYBER SECURITY
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO
Network Protocols & Architecture LAB
EXPERIMENT # 15**



```

AR1
AR1
[23BSCYS006-AR-1]dis ip int br
*down: administratively down
^down: standby
(l): loopback
(s): spoofing
The number of interface that is UP in Physical is 4
The number of interface that is DOWN in Physical is 0
The number of interface that is UP in Protocol is 4
The number of interface that is DOWN in Protocol is 0

Interface                               IP Address/Mask      Physical
GigabitEthernet0/0/0                   172.16.30.1/24       up
GigabitEthernet0/0/1                   172.16.10.1/24       up
GigabitEthernet0/0/2                   172.16.20.1/24       up
NULL0                                  unassigned            up

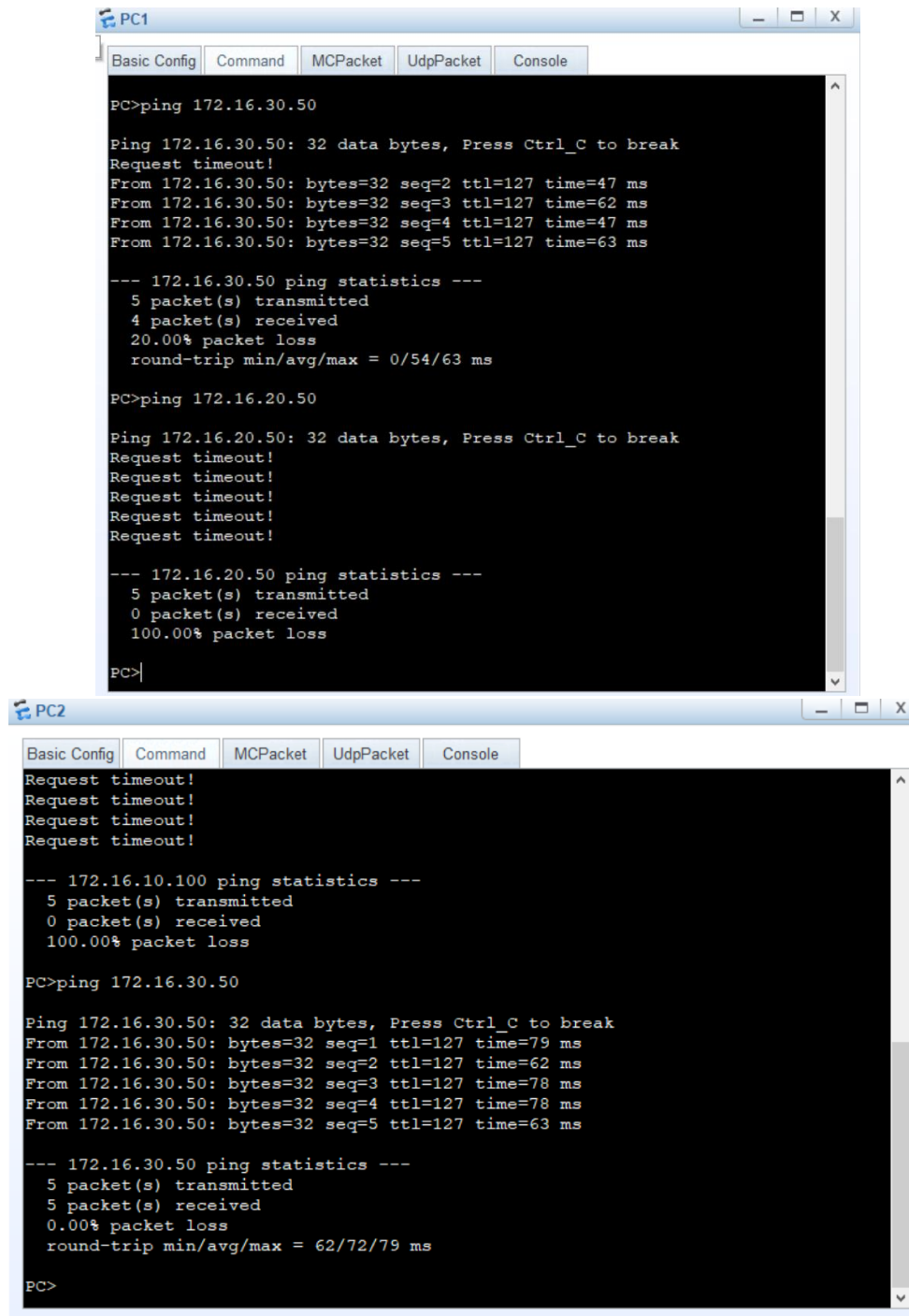
[23BSCYS006-AR-1]dis ac
[23BSCYS006-AR-1]dis acl ?
    INTEGER<2000-2999> Basic access-list(add to current using ru
    INTEGER<3000-3999> Advanced access-list(add to current using
    INTEGER<4000-4999> Specify a L2 acl group
    all                    All of ACL
    ipv6                  ACL IPv6
    name                   Specify a named ACL
    resource               ACL resources info
    vlan-division          Vlan section division
[23BSCYS006-AR-1]dis acl a
[23BSCYS006-AR-1]dis acl all
Total quantity of nonempty ACL number is 1

Advanced ACL 3000, 1 rule
Acl's step is 5
rule 5 deny ip destination 172.16.10.100 0 (21 matches)

[23BSCYS006-AR-1]

```

**DEPARTMENT OF TELECOMMUNICATION ENGINEERING BACHELOR OF
SCIENCE IN CYBER SECURITY
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO
Network Protocols & Architecture LAB
EXPERIMENT # 15**



The image displays two screenshots of a network simulator interface, likely Packet Tracer, showing the results of ping commands executed from two different PCs.

PC1 Console Output:

```
PC>ping 172.16.30.50

Ping 172.16.30.50: 32 data bytes, Press Ctrl_C to break
Request timeout!
From 172.16.30.50: bytes=32 seq=2 ttl=127 time=47 ms
From 172.16.30.50: bytes=32 seq=3 ttl=127 time=62 ms
From 172.16.30.50: bytes=32 seq=4 ttl=127 time=47 ms
From 172.16.30.50: bytes=32 seq=5 ttl=127 time=63 ms

--- 172.16.30.50 ping statistics ---
 5 packet(s) transmitted
 4 packet(s) received
20.00% packet loss
round-trip min/avg/max = 0/54/63 ms

PC>ping 172.16.20.50

Ping 172.16.20.50: 32 data bytes, Press Ctrl_C to break
Request timeout!
Request timeout!
Request timeout!
Request timeout!
Request timeout!

--- 172.16.20.50 ping statistics ---
 5 packet(s) transmitted
 0 packet(s) received
100.00% packet loss

PC>|
```

PC2 Console Output:

```
Request timeout!
Request timeout!
Request timeout!
Request timeout!

--- 172.16.10.100 ping statistics ---
 5 packet(s) transmitted
 0 packet(s) received
100.00% packet loss

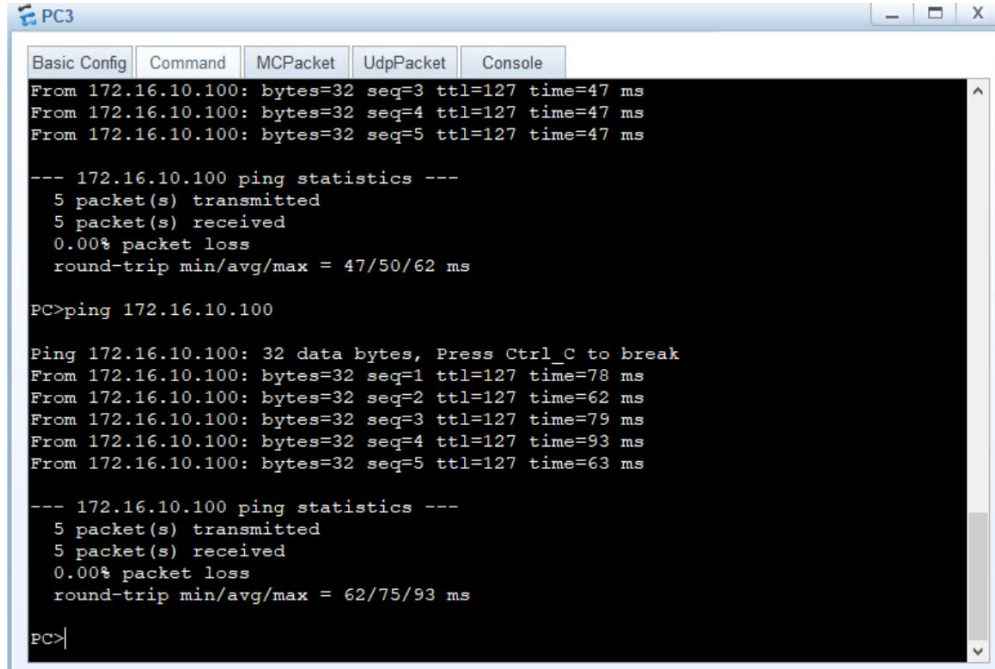
PC>ping 172.16.30.50

Ping 172.16.30.50: 32 data bytes, Press Ctrl_C to break
From 172.16.30.50: bytes=32 seq=1 ttl=127 time=79 ms
From 172.16.30.50: bytes=32 seq=2 ttl=127 time=62 ms
From 172.16.30.50: bytes=32 seq=3 ttl=127 time=78 ms
From 172.16.30.50: bytes=32 seq=4 ttl=127 time=78 ms
From 172.16.30.50: bytes=32 seq=5 ttl=127 time=63 ms

--- 172.16.30.50 ping statistics ---
 5 packet(s) transmitted
 5 packet(s) received
 0.00% packet loss
round-trip min/avg/max = 62/72/79 ms

PC>
```


**DEPARTMENT OF TELECOMMUNICATION ENGINEERING BACHELOR OF
SCIENCE IN CYBER SECURITY
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO
Network Protocols & Architecture LAB
EXPERIMENT # 15**



The screenshot shows a PC3 console window with a tabbed interface. The 'Console' tab is active, displaying the following text:

```
Basic Config | Command | MCPacket | UdpPacket | Console
From 172.16.10.100: bytes=32 seq=3 ttl=127 time=47 ms
From 172.16.10.100: bytes=32 seq=4 ttl=127 time=47 ms
From 172.16.10.100: bytes=32 seq=5 ttl=127 time=47 ms

--- 172.16.10.100 ping statistics ---
 5 packet(s) transmitted
 5 packet(s) received
 0.00% packet loss
 round-trip min/avg/max = 47/50/62 ms

PC>ping 172.16.10.100

Ping 172.16.10.100: 32 data bytes, Press Ctrl C to break
From 172.16.10.100: bytes=32 seq=1 ttl=127 time=78 ms
From 172.16.10.100: bytes=32 seq=2 ttl=127 time=62 ms
From 172.16.10.100: bytes=32 seq=3 ttl=127 time=79 ms
From 172.16.10.100: bytes=32 seq=4 ttl=127 time=93 ms
From 172.16.10.100: bytes=32 seq=5 ttl=127 time=63 ms

--- 172.16.10.100 ping statistics ---
 5 packet(s) transmitted
 5 packet(s) received
 0.00% packet loss
 round-trip min/avg/max = 62/75/93 ms

PC>|
```